



Mi Wallet

Tu billetera digital segura

Alke Wallet

Evolucion del modelo de datos

Este documento reúne las tres etapas de evolucion del modelo de datos de Alke Wallet, desde el modelo recibido originalmente, pasando por una mejora incremental (Approach), hasta una version normalizada en Tercera Forma Normal (Scalable). Cada seccion incluye el modelo conceptual, el modelo logico, el script SQL correspondiente y una comparativa con la version anterior.

Contenido

Alke Wallet.....2

1.1 Modelo Received (version inicial).....4

1.1.1 Modelo Conceptual.....4

1.1.2 Propósito del modelo4

1.1.3 Entidades y atributos.....4

1.2 Modelo conceptual — entidades y atributos5

 Usuario.....5

 Moneda5

 Transacción.....5

1.3 Relaciones entre entidades5

1.4 Diagrama Entidad Relación6

1.5 Modelo lógico — script SQL.....6

1.6 Limitaciones detectadas8

2.1 Modelo Approach (mejora incremental)9

1.1.1 Propósito del modelo9

1.1.2 Modelo conceptual — entidades y atributos9

 User (Usuario).....9

 Currency (Moneda).....9

 Transaction (Transacción).....9

2.2 Relaciones entre entidades9

2.3 Diagrama Entidad Relación10

2.5 Mejoras respecto al modelo Received.....12

3.1 Modelo Scalable (3FN).....13

1.1.1 Propósito del modelo13

1.1.2 Modelo conceptual — entidades y atributos13

 User (Usuario).....13

 Currency (Moneda).....13

 Account (Cuenta) — Nueva entidad13

 Transaction (Transacción).....14

3.2 Relaciones entre entidades14

3.3 Diagrama Entidad-Relación (ER)15

3.4 Modelo logico — script SQL.....16

3.5 Reglas de negocio – Validaciones de integridad.....17

3.6 Comparativa Approach -> Scalable17

 Mejoras respecto al modelo Approach.....17

3.7 Limitaciones resueltas y pendientes18

4.1 Consultas SQL19

 Captura de la creación de la Base de datos19

 Consulta para obtener el nombre de la moneda elegida por un usuario específico.19

 Captura de un resultado JOIN Exitosos.19

 Sub-Consulta para obtener todas las transacciones registradas por usuario.....20

 vista que muestre el top-5 de usuarios con mayor saldo.21

 Total de transacciones por moneda.....21

 Inserta registros de prueba en usuario, moneda y transacción.21

 Actualizar el saldo de un usuario luego de una transacción.22

 Captura de la consola mostrando un COMMIT exitoso.....23

 Simular un error de integridad referencial u revertir la operación.24

 Modificar la tabla usuario para añadir la fecha de creación usando ALTER TABLE.25

Conclusion general26

1.1 Modelo Received (version inicial)

Modelo de base de datos original entregado como punto de partida del proyecto **AlkeWallet**.

1.1. Modelo Conceptual

Modelo conceptual inicial recibido para el sistema **AlkeWallet**, identificando las entidades, atributos, relaciones y las principales decisiones de diseño. Sirve como base para comprender las limitaciones del modelo original y las mejoras propuestas en versiones posteriores.

1.2. Propósito del modelo

El modelo busca representar un sistema de wallet digital donde los usuarios pueden realizar transacciones financieras entre sí. La versión inicial se enfoca en la gestión básica de usuarios, monedas y movimientos, sin establecer relaciones complejas entre las entidades.

1.3. Entidades y atributos

Entidad	Descripción	Atributos clave
Usuario	Usuarios registrados en el sistema.	user_id (PK), nombre, correo_electronico, contraseña, saldo
Moneda	Catálogo de divisas admitidas.	currency_id (PK), currency_name, currency_symbol
Transacción	Registro de transferencias entre usuarios.	transaction_id (PK), importe, transaction_date, receiver_user_id (FK), sender_user_id (FK)

1.2 Modelo conceptual – entidades y atributos

Usuario

Representa a cada usuario registrado en el sistema.

Atributo	Tipo	Restricción	Descripción
user_id	INT	PK	Identificador único del usuario
nombre	VARCHAR(150)	NULL	Nombre completo
correo_electronico	VARCHAR(150)	NULL	Correo (sin UNIQUE en este modelo)
contrasena	VARCHAR(255)	NULL	Contraseña sin Hash.
saldo	INT	NULL	Saldo en la moneda base (sin decimales)

Moneda

Representa las divisas disponibles en el wallet digital.

Atributo	Tipo	Restriccion	Descripcion
currency_id	INT	PK	Identificador unico de la moneda
currency_name	VARCHAR(50)	NULL	Nombre de la moneda
currency_symbol	VARCHAR(50)	NULL	Símbolo monetario

Transacción

Registra las transferencias de dinero entre usuarios.

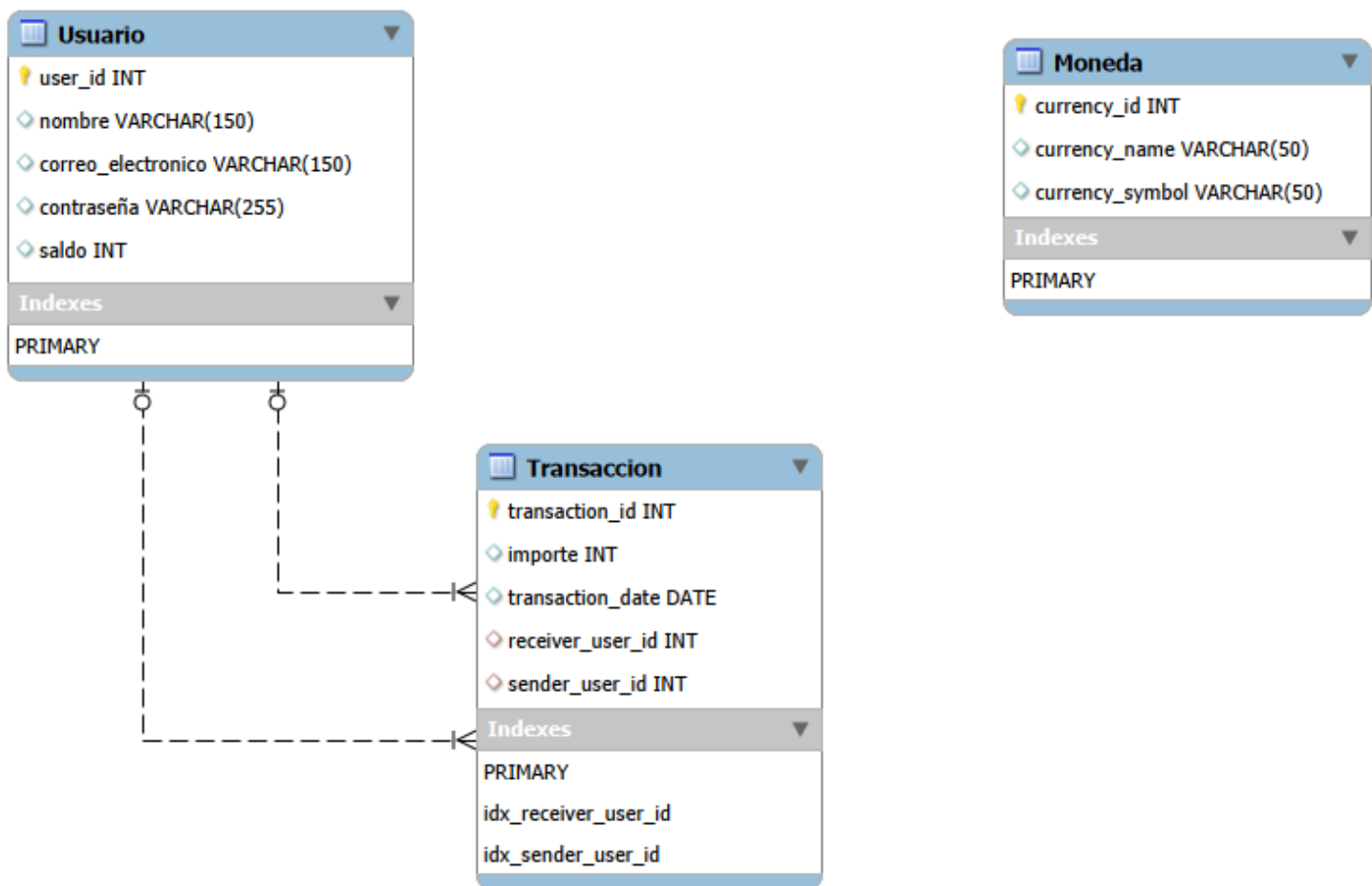
Atributo	Tipo	Restriccion	Descripcion
transaction_id	INT	PK	Identificador unico
importe	INT	NULL	Monto transferido (sin decimales)
transaction_date	DATE	NULL	Fecha de la operacion
receiver_user_id	INT	FK -> Usuario, NULL	Usuario receptor
sender_user_id	INT	FK -> Usuario, NULL	Usuario emisor

La tabla Moneda no tiene ninguna relación con Usuario ni Transacción: queda como catalogo aislado.

1.3 Relaciones entre entidades

Relación	Cardinalidad	PK/FK	Descripción
Usuario → Transacción (sender)	1 : N	sender_user_id → user_id	Un usuario puede enviar muchas transacciones. Cada transacción tiene un único emisor.
Usuario → Transacción (receiver)	1 : N	receiver_user_id → user_id	Un usuario puede recibir muchas transacciones. Cada transacción tiene un único receptor.
Moneda → (ninguna)	—	—	La entidad Moneda está desconectada: no se relaciona con Usuario ni Transacción.

1.4 Diagrama Entidad Relación



1.5 Modelo lógico – script SQL

[Script - Github](#)

```
1  -- -----
2  -- Schema AlkeWallet
3  -- -----
4  CREATE SCHEMA IF NOT EXISTS `AlkeWallet` DEFAULT CHARACTER SET utf8 COLLATE utf8_bin ;
5  USE `AlkeWallet` ;
6
7  -- -----
8  -- Table `AlkeWallet`.`Usuario`
9  -- -----
10 DROP TABLE IF EXISTS `AlkeWallet`.`Usuario` ;
11
12 CREATE TABLE IF NOT EXISTS `AlkeWallet`.`Usuario` (
13   `user_id` INT NOT NULL,
14   `nombre` VARCHAR(150) NULL,
15   `correo_electronico` VARCHAR(150) NULL,
16   `contraseña` VARCHAR(255) NULL,
17   `saldo` INT NULL,
18   PRIMARY KEY (`user_id`))
19 ENGINE = InnoDB;
20
21
22 -- -----
23 -- Table `AlkeWallet`.`Moneda`
24 -- -----
25 DROP TABLE IF EXISTS `AlkeWallet`.`Moneda` ;
26
27 CREATE TABLE IF NOT EXISTS `AlkeWallet`.`Moneda` (
28   `currency_id` INT NOT NULL,
29   `currency_name` VARCHAR(50) NULL,
30   `currency_symbol` VARCHAR(50) NULL,
31   PRIMARY KEY (`currency_id`))
32 ENGINE = InnoDB;
33
34
35 -- -----
36 -- Table `AlkeWallet`.`Transaccion`
37 -- -----
38 DROP TABLE IF EXISTS `AlkeWallet`.`Transaccion` ;
39
40 CREATE TABLE IF NOT EXISTS `AlkeWallet`.`Transaccion` (
41   `transaction_id` INT NOT NULL,
42   `importe` INT NULL,
43   `transaction_date` DATE NULL,
44   `receiver_user_id` INT NULL,
45   `sender_user_id` INT NULL,
46   PRIMARY KEY (`transaction_id`),
47   CONSTRAINT `fk_transaction_receiver_user_id`
48     FOREIGN KEY (`receiver_user_id`)
49     REFERENCES `AlkeWallet`.`Usuario` (`user_id`)
50     ON DELETE NO ACTION
51     ON UPDATE NO ACTION,
52   CONSTRAINT `fk_transaction_sender_user_id`
53     FOREIGN KEY (`sender_user_id`)
54     REFERENCES `AlkeWallet`.`Usuario` (`user_id`)
55     ON DELETE NO ACTION
56     ON UPDATE NO ACTION)
57 ENGINE = InnoDB;
```

1.6 Limitaciones detectadas

Este modelo fue recibido como punto de partida y presenta varias limitaciones que deben corregirse en versiones posteriores:

Área	Problema	Impacto
Tipos de datos	saldo e importe son INT	No permite valores decimales (centavos).
Integridad	Campos NULL permitidos en casi todas las columnas	Posibles registros incompletos.
Identificadores	PK sin AUTO_INCREMENT	Asignación manual de IDs propensa a errores.
Relaciones	Moneda desconectada de Usuario y Transaccion	No se puede asociar una transacción a una divisa ni saber la moneda preferida de un usuario.
Acción referencial	ON DELETE NO ACTION	Eliminación de usuario deja transacciones huérfanas.
Rendimiento	Sin índices adicionales	Consultas por sender_user_id o receiver_user_id serán lentas con grandes volúmenes de datos.

2.1 Modelo Approach (mejora incremental)

1.1. Propósito del modelo

El modelo Approach corrige las principales limitaciones: integridad de datos, tipos adecuados para montos, relación con moneda y rendimiento. Se mantiene la simplicidad de tres entidades, pero con mejoras significativas que lo hacen apto para un entorno productivo.

1.2. Modelo conceptual – entidades y atributos

User (Usuario)

Representa a cada usuario registrado en el sistema.

Atributo	Tipo	Restricciones	Descripción
user_id	INT	PK, AUTO_INCREMENT	Identificador único del usuario.
username	VARCHAR(100)	NOT NULL	Nombre completo del usuario.
email	VARCHAR(150)	NOT NULL, UNIQUE	Correo electrónico (único).
password	VARCHAR(255)	NOT NULL	Hash de la contraseña.
current_balance	DECIMAL(15,2)	NOT NULL, DEFAULT 0	Saldo disponible en la moneda base del sistema.

Currency (Moneda)

Catálogo de divisas admitidas en el monedero.

Atributo	Tipo	Restricciones	Descripción
currency_id	INT	PK, AUTO_INCREMENT	Identificador único de la moneda.
currency_name	VARCHAR(50)	NOT NULL, UNIQUE	Nombre de la moneda (ej. "Peso Chileno").
currency_symbol	VARCHAR(5)	NOT NULL, UNIQUE	Símbolo monetario (ej. "\$", "€").

Transaction (Transacción)

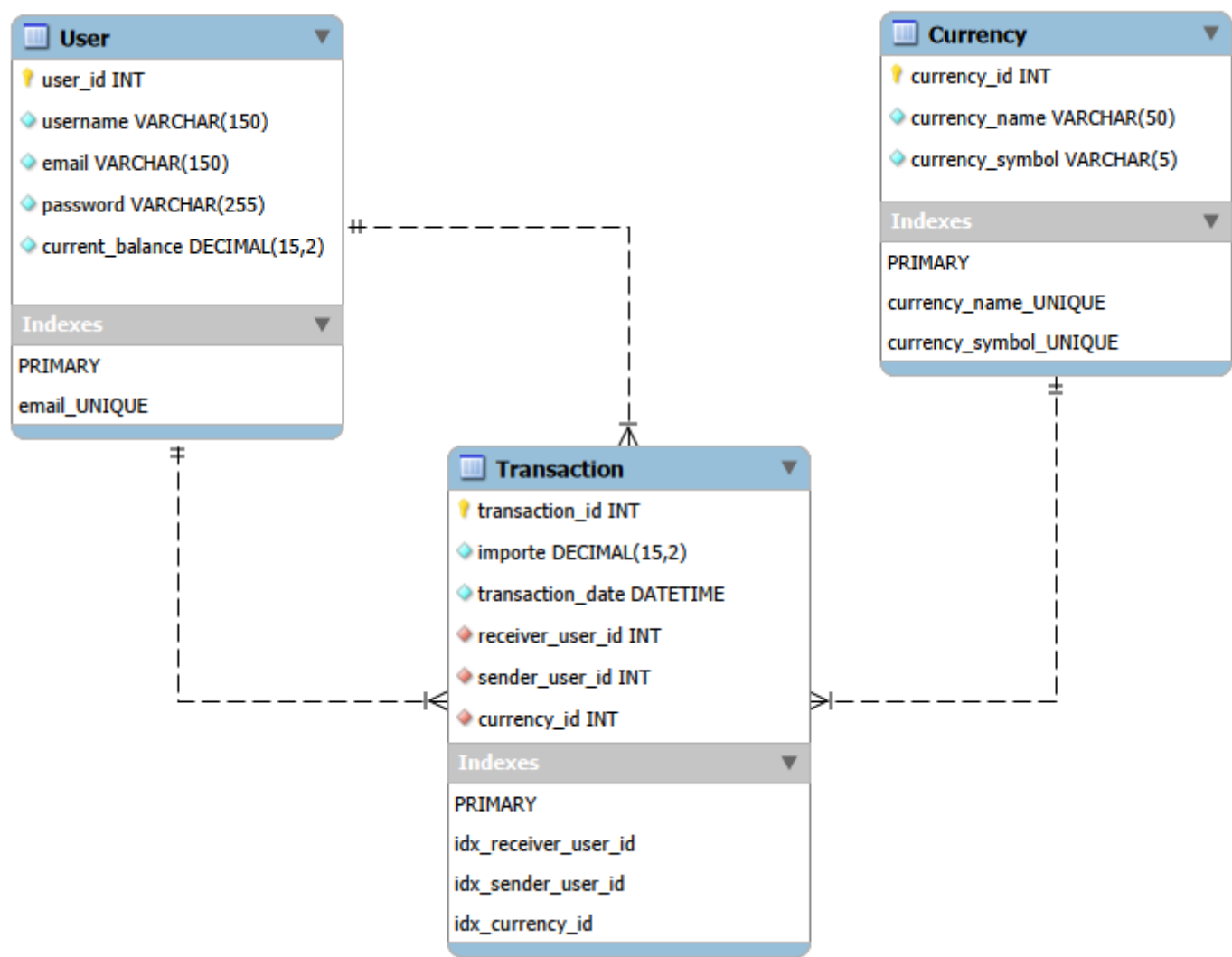
Registra cada transferencia de dinero entre usuarios.

Atributo	Tipo	Restricciones	Descripción
transaction_id	INT	PK, AUTO_INCREMENT	Identificador único de la transacción.
importe	DECIMAL(15,2)	NOT NULL	Monto transferido (con centavos).
transaction_date	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Fecha y hora exacta de la operación.
sender_user_id	INT	NOT NULL, FK → User(user_id)	Usuario que envía el dinero.
receiver_user_id	INT	NOT NULL, FK → User(user_id)	Usuario que recibe el dinero.
currency_id	INT	NOT NULL, FK → Currency(currency_id)	Moneda en que se realizó la transacción.

2.2 Relaciones entre entidades

Relación	Cardinalidad	PK/FK	Descripción
User → Transaction (sender)	1 : N	sender_user_id → user_id	Un usuario puedeenviar muchas transacciones. Cada transacción tiene un único emisor.
User → Transaction (receiver)	1 : N	receiver_user_id → user_id	Un usuario puederecibir muchas transacciones. Cada transacción tiene un único receptor.
Currency → Transaction	1 : N	currency_id → currency_id	Una moneda puede aparecer enmuchas transacciones. Cada transacción usa una sola moneda.

2.3 Diagrama Entidad Relación



2.4 Modelo logico – script SQL

[Script -Github](#)

```
1  -- -----
2  -- Table `AlkeWallet`.`User`
3  -- -----
4  DROP TABLE IF EXISTS `AlkeWallet`.`User` ;
5
6  CREATE TABLE IF NOT EXISTS `AlkeWallet`.`User` (
7    `user_id` INT NOT NULL AUTO_INCREMENT,
8    `username` VARCHAR(150) NOT NULL,
9    `email` VARCHAR(150) NOT NULL,
10   `password` VARCHAR(255) NOT NULL,
11   `current_balance` DECIMAL(15,2) NOT NULL,
12   PRIMARY KEY (`user_id`),
13   UNIQUE INDEX `email_UNIQUE` (`email` ASC) VISIBLE)
14  ENGINE = InnoDB;
15
16
17  -- -----
18  -- Table `AlkeWallet`.`Currency`
19  -- -----
20  DROP TABLE IF EXISTS `AlkeWallet`.`Currency` ;
21
22  CREATE TABLE IF NOT EXISTS `AlkeWallet`.`Currency` (
23    `currency_id` INT NOT NULL AUTO_INCREMENT,
24    `currency_name` VARCHAR(50) NOT NULL,
25    `currency_symbol` VARCHAR(5) NOT NULL,
26    PRIMARY KEY (`currency_id`),
27    UNIQUE INDEX `currency_name_UNIQUE` (`currency_name` ASC) VISIBLE,
28    UNIQUE INDEX `currency_symbol_UNIQUE` (`currency_symbol` ASC) VISIBLE)
29  ENGINE = InnoDB;
30
31
32  -- -----
33  -- Table `AlkeWallet`.`Transaction`
34  -- -----
35  DROP TABLE IF EXISTS `AlkeWallet`.`Transaction` ;
36
37  CREATE TABLE IF NOT EXISTS `AlkeWallet`.`Transaction` (
38    `transaction_id` INT NOT NULL AUTO_INCREMENT,
39    `importe` DECIMAL(15,2) NOT NULL,
40    `transaction_date` DATETIME NOT NULL,
41    `receiver_user_id` INT NOT NULL,
42    `sender_user_id` INT NOT NULL,
43    `currency_id` INT NOT NULL,
44    PRIMARY KEY (`transaction_id`),
45    INDEX `idx_receiver_user_id` (`receiver_user_id` ASC) VISIBLE,
46    INDEX `idx_sender_user_id` (`sender_user_id` ASC) VISIBLE,
47    INDEX `idx_currency_id` (`currency_id` ASC) VISIBLE,
48    CONSTRAINT `fk_transaction_receiver_user_id`
49      FOREIGN KEY (`receiver_user_id`)
50      REFERENCES `AlkeWallet`.`User` (`user_id`)
51      ON DELETE CASCADE
52      ON UPDATE CASCADE,
53    CONSTRAINT `fk_transaction_sender_user_id`
54      FOREIGN KEY (`sender_user_id`)
55      REFERENCES `AlkeWallet`.`User` (`user_id`)
56      ON DELETE CASCADE
57      ON UPDATE CASCADE,
58    CONSTRAINT `fk_transaction_currency_id`
59      FOREIGN KEY (`currency_id`)
60      REFERENCES `AlkeWallet`.`Currency` (`currency_id`)
61      ON DELETE CASCADE
62      ON UPDATE CASCADE)
63  ENGINE = InnoDB;
```

2.5 Mejoras respecto al modelo Received

La siguiente tabla compara el modelo Approach con el modelo inicial (Received), destacando las mejoras aplicadas:

Aspecto	Modelo Received	Modelo Approach	Beneficio
Identificadores	INT NOT NULL (manual)	INT AUTO_INCREMENT	Simplifica inserciones y evita errores de duplicación.
Montos y saldos	INT (sin decimales)	DECIMAL(15,2)	Permite centavos y precisión financiera.
Campos obligatorios	NULL permitido	NOT NULL en todos los campos	Garantiza integridad mínima de los datos.
Correo único	Sin restricción	UNIQUE(email)	Evita duplicados de correo electrónico.
Moneda única	Sin restricción	UNIQUE(currency_name), UNIQUE(currency_symbol)	Previene nombres o símbolos duplicados.
Relación con moneda	Currency aislada	currency_id (FK) en Transaction	Asocia cada transacción a su divisa.
Fecha de transacción	DATE (solo día)	DATETIME (día + hora)	Permite auditoría precisa y ordenamiento cronológico.
Índices de rendimiento	Solo PK implícitas	Índices sender_user_id, receiver_user_id, currency_id, transaction_date	Acelera consultas frecuentes.
Acción referencial	ON DELETE NO ACTION	ON DELETE CASCADE / ON UPDATE CASCADE	Mantiene consistencia al eliminar usuarios.

3.1 Modelo Scalable (3FN)

1.1. Propósito del modelo

El modelo Scalable representa el esquema final de AlkeWallet, alcanzando la Tercera Forma Normal (3FN). Su principal innovación es la introducción de la entidad Account, que permite a los usuarios tener múltiples saldos en diferentes monedas y desacopla el saldo de la tabla User. Este modelo está diseñado para entornos productivos que requieran flexibilidad, integridad y auditoría avanzada.

1.2. Modelo conceptual – entidades y atributos

User (Usuario)

Representa a cada persona registrada en el sistema. **Ya no almacena saldo**; ese dato se traslada a Account.

Atributo	Tipo	Restricciones	Descripción
user_id	INT	PK, AUTO_INCREMENT	Identificador único del usuario.
user_name	VARCHAR(100)	NOT NULL	Nombre completo del usuario.
email	VARCHAR(150)	NOT NULL, UNIQUE	Correo electrónico (único).
password	VARCHAR(255)	NOT NULL	Hash de la contraseña.

Currency (Moneda)

Catálogo de divisas admitidas en el sistema.

Atributo	Tipo	Restricciones	Descripción
currency_id	INT	PK, AUTO_INCREMENT	Identificador único de la moneda.
currency_name	VARCHAR(50)	NOT NULL, UNIQUE	Nombre de la moneda (ej. "Peso Chileno").
currency_symbol	VARCHAR(5)	NOT NULL, UNIQUE	Símbolo monetario (ej. "\$", "€").

Account (Cuenta) – Nueva entidad

Representa el saldo de un **usuario** en una **moneda** específica. Es una entidad débil que depende de User y Currency.

Atributo	Tipo	Restricciones	Descripción
account_id	INT	PK, AUTO_INCREMENT	Identificador único de la cuenta.
user_id	INT	NOT NULL, FK → User(user_id)	Usuario propietario de la cuenta.
currency_id	INT	NOT NULL, FK → Currency(currency_id)	Moneda de la cuenta.
current_balance	DECIMAL(15,2)	NOT NULL, DEFAULT 0, CHECK (balance >= 0)	Saldo actual en esa moneda.
is_default	BOOLEAN	NOT NULL, DEFAULT FALSE	Indica si es la cuenta principal del usuario.

Transaction (Transacción)
Registra transferencias de dinero **entre cuentas** (no directamente entre usuarios).

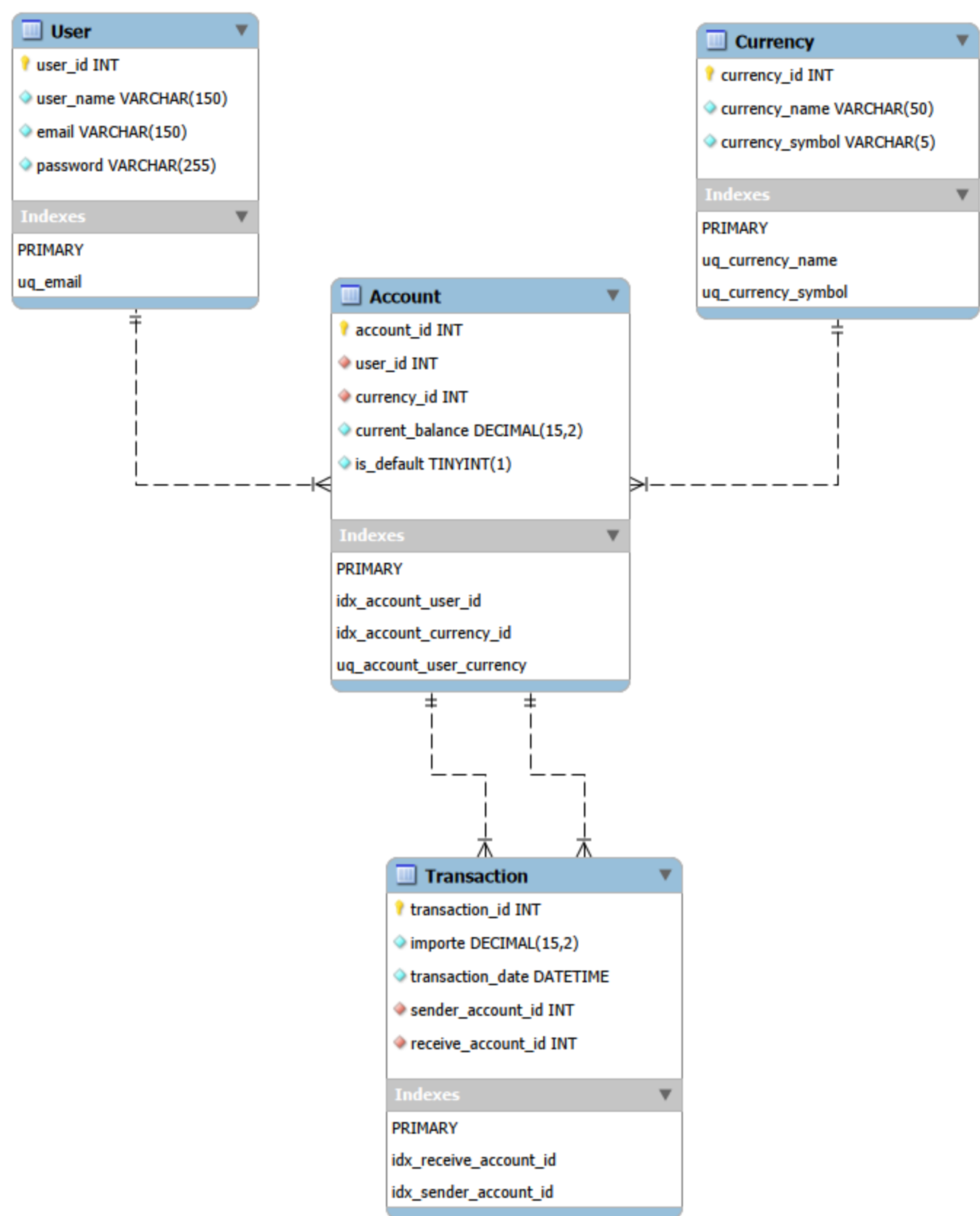
Atributo	Tipo	Restricciones	Descripción
transaction_id	INT	PK, AUTO_INCREMENT	Identificador único de la transacción.
importe	DECIMAL(15,2)	NOT NULL, CHECK (importe > 0)	Monto transferido.
transaction_date	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Fecha y hora exacta de la operación.
sender_account_id	INT	NOT NULL, FK → Account(account_id)	Cuenta que envía el dinero.
receive_account_id	INT	NOT NULL, FK → Account(account_id)	Cuenta que recibe el dinero.

Regla de negocio: sender_account_id y receive_account_id deben ser distintos (CHECK (sender_account_id <> receive_account_id)). La moneda de la transacción es implícita (la de las cuentas involucradas).

3.2 Relaciones entre entidades

Relación	Cardinalidad	PK/FK	Descripción
User → Account	1 : N	user_id → user_id	Un usuario puede tener muchas cuentas (una por moneda). Cada cuenta pertenece a un único usuario.
Currency → Account	1 : N	currency_id → currency_id	Una moneda puede estar asociada a muchas cuentas (muchos usuarios la poseen). Cada cuenta usa una sola moneda.
Account → Transaction (emisor)	1 : N	sender_account_id → account_id	Una cuenta puede enviar muchas transacciones. Cada transacción tiene una única cuenta emisora.
Account → Transaction (receptor)	1 : N	receive_account_id → account_id	Una cuenta puede recibir muchas transacciones. Cada transacción tiene una única cuenta receptora.

3.3 Diagrama Entidad-Relación (ER)



3.4 Modelo logico – script SQL

[Script -Github](#)

```
1  -----
2  -- Schema AlkeWallet
3  -----
4  CREATE SCHEMA IF NOT EXISTS `AlkeWallet` DEFAULT CHARACTER SET utf8 COLLATE utf8_bin ;
5  USE `AlkeWallet` ;
6
7  -----
8  -- Table `AlkeWallet`.`User`
9  -----
10 DROP TABLE IF EXISTS `AlkeWallet`.`User` ;
11
12 CREATE TABLE IF NOT EXISTS `AlkeWallet`.`User` (
13   `user_id` INT NOT NULL AUTO_INCREMENT,
14   `user_name` VARCHAR(150) NOT NULL,
15   `email` VARCHAR(150) NOT NULL,
16   `password` VARCHAR(255) NOT NULL,
17   PRIMARY KEY (`user_id`),
18   UNIQUE INDEX `uq_email` (`email` ASC) VISIBLE)
19 ENGINE = InnoDB;
20
21
22 -----
23 -- Table `AlkeWallet`.`Currency`
24 -----
25 DROP TABLE IF EXISTS `AlkeWallet`.`Currency` ;
26
27 CREATE TABLE IF NOT EXISTS `AlkeWallet`.`Currency` (
28   `currency_id` INT NOT NULL AUTO_INCREMENT,
29   `currency_name` VARCHAR(50) NOT NULL,
30   `currency_symbol` VARCHAR(5) NOT NULL,
31   PRIMARY KEY (`currency_id`),
32   UNIQUE INDEX `uq_currency_name` (`currency_name` ASC) VISIBLE,
33   UNIQUE INDEX `uq_currency_symbol` (`currency_symbol` ASC) VISIBLE)
34 ENGINE = InnoDB;
35
36
37 -----
38 -- Table `AlkeWallet`.`Account`
39 -----
40 DROP TABLE IF EXISTS `AlkeWallet`.`Account` ;
41
42 CREATE TABLE IF NOT EXISTS `AlkeWallet`.`Account` (
43   `account_id` INT NOT NULL AUTO_INCREMENT,
44   `user_id` INT NOT NULL,
45   `currency_id` INT NOT NULL,
46   `current_balance` DECIMAL(15,2) NOT NULL DEFAULT 0,
47   `is_default` TINYINT(1) NOT NULL DEFAULT 0,
48   PRIMARY KEY (`account_id`),
49   UNIQUE INDEX `uq_account_user_currency` (`user_id` ASC, `currency_id` ASC) VISIBLE,
50   CONSTRAINT `fk_account_user_id`
51     FOREIGN KEY (`user_id`)
52     REFERENCES `AlkeWallet`.`User` (`user_id`)
53     ON DELETE RESTRICT
54     ON UPDATE RESTRICT,
55   CONSTRAINT `fk_account_currency_id`
56     FOREIGN KEY (`currency_id`)
57     REFERENCES `AlkeWallet`.`Currency` (`currency_id`)
58     ON DELETE RESTRICT
59     ON UPDATE RESTRICT)
60 ENGINE = InnoDB;
61
62
63 -----
64 -- Table `AlkeWallet`.`Transaction`
65 -----
66 DROP TABLE IF EXISTS `AlkeWallet`.`Transaction` ;
67
68 CREATE TABLE IF NOT EXISTS `AlkeWallet`.`Transaction` (
69   `transaction_id` INT NOT NULL AUTO_INCREMENT,
70   `importe` DECIMAL(15,2) NOT NULL,
71   `transaction_date` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
72   `sender_account_id` INT NOT NULL,
73   `receive_account_id` INT NOT NULL,
74   PRIMARY KEY (`transaction_id`),
75   INDEX `idx_receive_account_id` (`receive_account_id` ASC) VISIBLE,
76   INDEX `idx_sender_account_id` (`sender_account_id` ASC) VISIBLE,
77   CONSTRAINT `fk_transaction_sender_account_id`
78     FOREIGN KEY (`sender_account_id`)
79     REFERENCES `AlkeWallet`.`Account` (`account_id`)
80     ON DELETE RESTRICT
81     ON UPDATE RESTRICT,
82   CONSTRAINT `fk_transaction_receive_account_id`
83     FOREIGN KEY (`receive_account_id`)
84     REFERENCES `AlkeWallet`.`Account` (`account_id`)
85     ON DELETE RESTRICT
86     ON UPDATE RESTRICT)
87 ENGINE = InnoDB;
```


3.5 Reglas de negocio – Validaciones de integridad

```
1  -----
2  -- Validaciones de integridad para Account
3  -----
4  ALTER TABLE `AlkeWallet`.`Account`
5  ADD CONSTRAINT `chk_account_balance_non_negative`
6  CHECK (`current_balance` >= 0);
7
8  -----
9  -- Validación de importe positivo en Transaction
10 -----
11 ALTER TABLE `AlkeWallet`.`Transaction`
12 ADD CONSTRAINT `chk_transaction_amount_positive`
13 CHECK (`importe` > 0);
14
15 -----
16 -- Validación de cuentas distintas en Transaction
17 -----
18 ALTER TABLE `AlkeWallet`.`Transaction`
19 ADD CONSTRAINT `chk_transaction_accounts_different`
20 CHECK (`sender_account_id` <> `receive_account_id`);
```

3.6 Comparativa Approach -> Scalable

Mejoras respecto al modelo Approach

La siguiente tabla resume los cambios estructurales y funcionales más importantes:

Aspecto	Modelo Approach	Modelo Scalable	Beneficio
Saldo del usuario	EnUser.current_balance (único)	EnAccount.current_balance (uno por moneda)	Permite múltiples divisas por usuario.
Relación User ↔ Currency	Indirecta (víaTransaction)	Directa (víaAccount)	Asociación explícita y normalizada.
Transacciones	Entre usuarios (sender_user_id, receiver_user_id)	Entre cuentas (sender_account_id, receiver_account_id)	La moneda se deduce de la cuenta, simplificando la lógica.
Acción referencial	ON DELETE CASCADE	ON DELETE RESTRICT	Evita borrar accidentalmente el historial financiero.
Validaciones de negocio	SinCHECK	CHECK (balance >= 0), CHECK (importe > 0), CHECK (sender <> receiver)	Garantiza reglas financieras básicas a nivel de BD.
Cuenta predeterminada	No existía	is_default en Account	Permite identificar la cuenta principal del usuario.
Normalización	2FN	3FN	Elimina dependencias funcionales, reduce redundancia.

3.7 Limitaciones resueltas y pendientes

Resueltas en Scalable

Soporte multi-moneda (tabla Account).

- Validaciones de negocio (CHECK constraints).
- Protección contra borrados accidentales (RESTRICT).
- Cuenta predeterminada (is_default).

Pendientes (fuera del alcance actual)

- Borrado lógico: No se implementa is_active en User; se recomienda como extensión.

4.1 Consultas SQL

Scripts -Github

Captura de la creación de la Base de datos

#	Time	Action	Message	Duration / Fetch
1	22:15:48	DROP DATABASE 'alkewallet'	4 row(s) affected	0.032 sec
2	22:15:57	DROP SCHEMA IF EXISTS 'AkeWallet'	0 row(s) affected, 1 warning(s): 1008 Can't drop database 'alkewallet'; database doesn't exist	0.000 sec

Consulta para obtener el nombre de la moneda elegida por un usuario específico.

Captura de un resultado JOIN Exitosos.

The screenshot shows the SQL Server Enterprise Manager interface. At the top, there are tabs for 'SQL File 6*', 'SQL File 7*', and 'SQL File 8*'. The 'SQL File 8*' tab is active, displaying a SQL query in a text editor. The query is as follows:

```

1 SELECT
2     u.`user_name`,
3     -- c.`currency_symbol` AS `simbolo_moneda`
4     c.`currency_name` AS `moneda_elegida`
5 FROM `User` u
6 LEFT JOIN `Account` a
7     ON u.`user_id` = a.`user_id` AND a.`is_default` = 1
8 LEFT JOIN `Currency` c
9     ON a.`currency_id` = c.`currency_id`
10 WHERE u.`user_id` = 11;

```

Below the query editor, the 'Results' pane shows the 'Result Grid' with two columns: 'user_name' and 'moneda_elegida'. The data row shows 'Ana Díaz' and 'Peso Chileno'. To the right of the grid is a 'Read Only' button.

At the bottom, the 'Output' pane shows the 'Action Output' for the query execution. It displays the following message:

```

# Time Action Message
1 22:18:57 SELECT u.`user_name`, -- c.`currency_symbol` AS `simbolo_moneda` c.`currency_name` AS `moneda_elegida` F... 1 row(s) returned

```

The 'Duration / Fetch' column shows '0.000 sec / 0.000 sec'.

Sub-Consulta para obtener todas las transacciones registradas por usuario.

```
1  -- sub-consultas para obtener el total de transacciones por usuario.
2  SELECT
3      u.user_id,
4      u.user_name,
5      COALESCE(conteo.total, 0) AS total_transacciones
6  FROM `User` u
7  LEFT JOIN (
8      SELECT
9          a.user_id,
10         COUNT(*) AS total
11      FROM `Transaction` t
12      INNER JOIN `Account` a
13          ON a.account_id = t.sender_account_id OR a.account_id = t.receive_account_id
14      GROUP BY a.user_id
15  ) AS conteo ON u.user_id = conteo.user_id
16  ORDER BY total_transacciones DESC;
```

vista que muestre el top-5 de usuarios con mayor saldo.



```
1  -- top-5 de usuarios con mayor saldo.
2  CREATE VIEW vw_top5_saldo_cuenta_default AS
3  SELECT
4      u.user_id,
5      u.user_name,
6      c.currency_name,
7      a.current_balance
8  FROM `Account` a
9  INNER JOIN `User` u ON a.user_id = u.user_id
10 INNER JOIN `Currency` c ON a.currency_id = c.currency_id
11 WHERE a.is_default = 1
12 ORDER BY a.current_balance DESC
13 LIMIT 5;
```

Total de transacciones por moneda



```
1  Total de transacciones por moneda (deducida de las cuentas)
2  SELECT
3      c.currency_name,
4      COUNT(t.transaction_id) AS total_transacciones,
5      SUM(t.importe) AS monto_total
6  FROM `Transaction` t
7  JOIN `Account` a_sender ON t.sender_account_id = a_sender.account_id
8  JOIN `Currency` c ON a_sender.currency_id = c.currency_id
9  GROUP BY c.currency_id, c.currency_name
10 ORDER BY total_transacciones DESC;
```

Inserta registros de prueba en usuario, moneda y transacción.

[Script - Github](#)

Actualizar el saldo de un usuario luego de una transacción.



```
1  -- Actualizar el saldo de un usuario luego de una transacción.
2  USE `AlkeWallet`;
3
4  START TRANSACTION;
5
6  -- 1. Descontar el importe de la cuenta emisora
7  UPDATE `Account`
8  SET current_balance = current_balance - 50000
9  WHERE account_id = 3;
10
11 -- 2. Sumar el importe a la cuenta receptora
12 UPDATE `Account`
13 SET current_balance = current_balance + 50000
14 WHERE account_id = 7;
15
16 -- 3. Registrar la transacción
17 INSERT INTO `Transaction` (importe, sender_account_id, receive_account_id)
18 VALUES (50000, 3, 7);
19
20 -- 4. Verificar que todo quedó correcto antes de confirmar
21 SELECT account_id, current_balance FROM `Account` WHERE account_id IN (3, 7);
22
23 -- 5. Si todo está bien, confirmar de forma permanente
24 COMMIT;
```

Captura de la consola mostrando un COMMIT exitoso.

The screenshot displays the SQL Server Enterprise Manager interface. At the top, the 'SQL File 8' tab is active, showing a SQL script with the following content:

```
1 -- Actualizar el saldo de un usuario luego de una transacción.
2 USE 'AlkeWallet';
3
4 START TRANSACTION;
5
6 -- 1. Descontar el importe de la cuenta emisora
7 UPDATE 'Account'
8 SET current_balance = current_balance - 50000
9 WHERE account_id = 3;
10
11 -- 2. Sumar el importe a la cuenta receptora
12 UPDATE 'Account'
13 SET current_balance = current_balance + 50000
14 WHERE account_id = 7;
15
16 -- 3. Registrar la transacción
17 INSERT INTO 'Transaction' (importe, sender_account_id, receive_account_id)
18 VALUES (50000, 3, 7);
19
20 -- 4. Verificar que todo quedó correcto antes de confirmar
21 SELECT account_id, current_balance FROM 'Account' WHERE account_id IN (3, 7);
22
23 -- 5. Si todo está bien, confirmar de forma permanente
24 COMMIT;
```

Below the script, the 'Result Grid' shows the output of the SELECT statement:

account_id	current_balance
3	304785.00
7	63912.00

The 'Output' pane at the bottom shows the execution log for the script:

#	Time	Action	Message	Duration / Fetch
1	22:18:57	SELECT u.'user_name', --c.'currency_symbol' AS 'simbolo_moneda' c.'currency_name' AS 'moneda_elegida' F...	1 row(s) returned	0.000 sec / 0.000 sec
2	22:26:16	USE 'AlkeWallet'	0 row(s) affected	0.000 sec
3	22:26:16	START TRANSACTION	0 row(s) affected	0.000 sec
4	22:26:16	UPDATE 'Account' SET current_balance = current_balance - 50000 WHERE account_id = 3	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.000 sec
5	22:26:16	UPDATE 'Account' SET current_balance = current_balance + 50000 WHERE account_id = 7	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.000 sec
6	22:26:16	INSERT INTO 'Transaction' (importe, sender_account_id, receive_account_id) VALUES (50000, 3, 7)	1 row(s) affected	0.000 sec
7	22:26:16	SELECT account_id, current_balance FROM 'Account' WHERE account_id IN (3, 7)	2 row(s) returned	0.000 sec / 0.000 sec
8	22:26:16	COMMIT	0 row(s) affected	0.000 sec

Simular un error de integridad referencial u revertir la operación.

```
1  -- Error de integridad referencial y revertir la operación.
2  -- Selecciona la base de datos AlkeWallet como la BD activa para esta sesión;
3  USE `AlkeWallet`;
4
5  -- Inicia una transacción: desactiva el autocommit para las sentencias que
6  -- siguen, agrupándolas como una única unidad "todo o nada".
7  START TRANSACTION;
8
9  -- Descuenta 20000 del saldo de la cuenta con account_id = 3 (existe).
10 -- Este cambio queda "pendiente", visible solo dentro de esta transacción.
11 UPDATE `Account`
12 SET current_balance = current_balance - 20000
13 WHERE account_id = 3;
14
15 -- Intenta sumar 20000 a la cuenta con account_id = 999 (NO existe).
16 -- No da error: el WHERE simplemente no encuentra ninguna fila que coincida,
17 -- por lo que MySQL reporta "0 rows affected" sin fallar.
18 UPDATE `Account`
19 SET current_balance = current_balance + 20000
20 WHERE account_id = 999;
21
22 -- Intenta registrar la transacción entre la cuenta 3 (emisora) y la cuenta
23 -- 999 (receptora, inexistente). Aquí SI falla: la FOREIGN KEY hacia Account
24 -- rechaza el INSERT con el error 1452, porque 999 no es un account_id válido.
25 INSERT INTO `Transaction` (importe, sender_account_id, receive_account_id)
26 VALUES (20000, 3, 999);
27
28 -- Revierte TODOS los cambios hechos desde el START TRANSACTION (en este
29 -- caso, el UPDATE a la cuenta 3), dejando la base de datos exactamente como
30 -- estaba antes de que empezara el bloque. Se ejecuta tras ver el error 1452.
31 ROLLBACK;
32
33 -- Verifica que el saldo de la cuenta 3 haya vuelto a su valor original,
34 -- confirmando que el ROLLBACK deshizo el descuento de 20000.
35 SELECT account_id, current_balance FROM `Account` WHERE account_id = 3;
36
37 -- Verifica que la transacción fallida nunca haya quedado registrada:
38 -- este SELECT debería devolver 0 filas, ya que el INSERT nunca se confirmó.
39 SELECT * FROM `Transaction` WHERE sender_account_id = 3 AND importe = 20000;
40
41 /* Actualizar al Saldo original de la cuenta.
42 UPDATE `Account`
43 SET current_balance = 354785.00
44 WHERE account_id = 3;
45 */
```


Modificar la tabla usuario para añadir la fecha de creación usando ALTER TABLE.



```
1  -- Modificar la tabla usuario para añadir la fecha de creación usando ALTER TABLE
2  ALTER TABLE `User`
3  ADD COLUMN `created_at` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP;
```

Conclusion general

La evolucion Received -> Approach -> Scalable se documenta de forma progresiva como un modelo de datos puede pasar de un esquema funcional pero con riesgos de integridad, a una versión corregida para producción con tipos de datos correctos y relaciones completas (Approach), y finalmente a una versión normalizada en 3FN que soporta múltiples monedas por usuario y protege el historial financiero de borrados accidentales (Scalable). Cada versión es independiente y ejecutable; la elección entre ellas depende de los requisitos reales del sistema.

Crafted by [whiterabbit](#) 